

Binary Search Tree (BST)

— A DS that supports dynamic set operations: Search, Min/Max, Successor, Predecessor, Insert, Delete

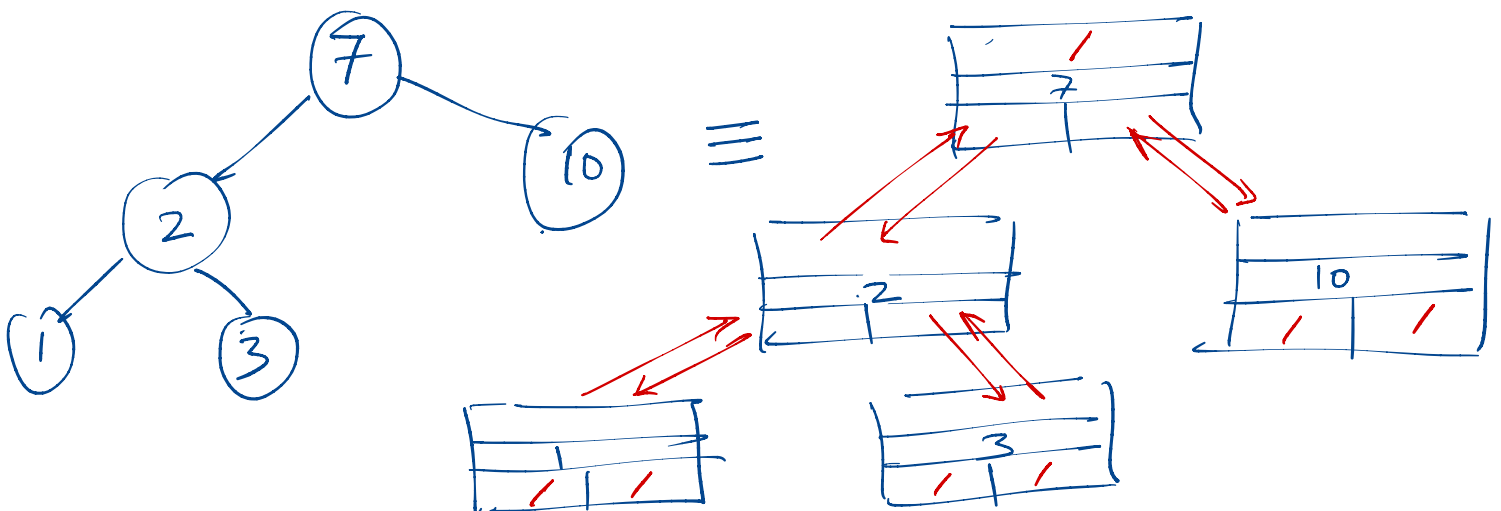
— Basic operations take time proportional to the height of the tree

— Attributes of a BST node:

- key ↗ value
- Left ↖ Pointer to root node of left subtree
- Right ↖ Pointer to root node of right subtree
- P → Pointer to the parent node

BST node :

P	
key	
left	right



Struct Node

{

int key;

Node *left;

Node *right;

Node *P;

};

— The root node $T.root$ is the only node whose parent is NIL

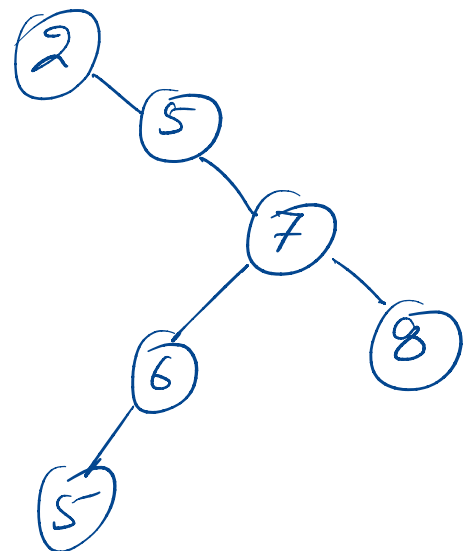
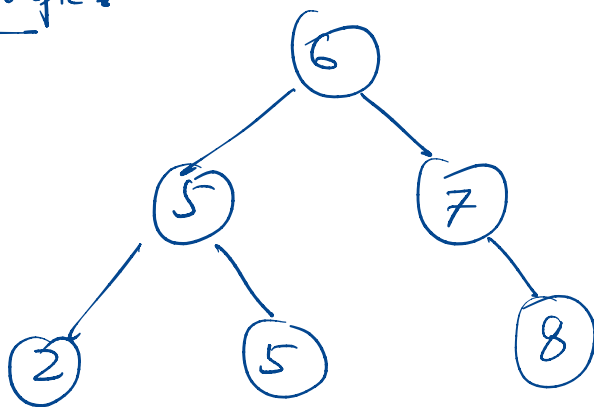
keys in a BST satisfy the following property:

" let x be a node in a BST. If y is a node in the left subtree of x , then $y.key \leq x.key$.

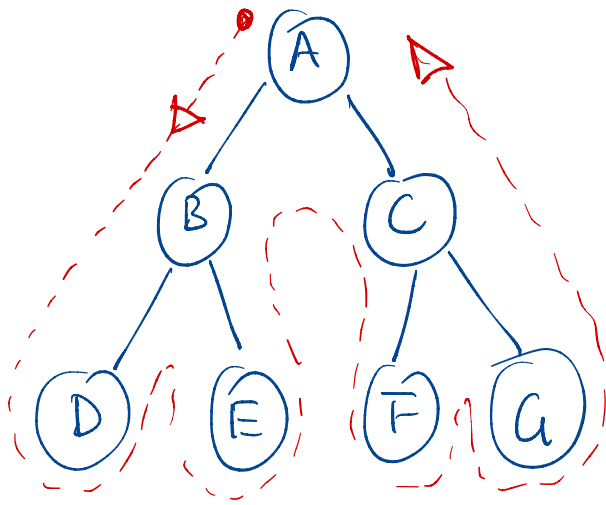
If y is a node in the right subtree of x then,
 $y.key \geq x.key$ "

→ Invariant of a BST

Example:



Tree traversals :



(I) Depth first search (DFS)
Let's say : Current Node - C

— Pre order walk **CLR**

- a) Visit current node (root)
- b) Explore the left subtree
- c) Explore the right subtree

A B D E C F G

— In order walk **LCR**

- a) Explore the left subtree
- b) visit the root node
- c) Explore the right subtree

D B E A F C G

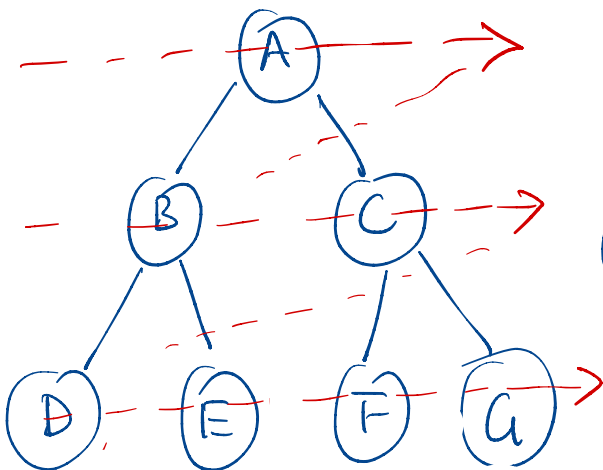
— Post order walk **LRC**

- a) Explore the left subtree
- b) Explore the right subtree
- c) Visit the root

D E B F G C A

(II) Breadth first search (BFS)
(level-order search)

A B C D E F G



BST traversal

— Possible to print all keys in a BST in

Sorted order by an In order walk
Due to the Property of BST

In order tree walk:

Perform inorder walk
on a subtree rooted
at node x

Inorder-Tree-Walk(x)

if $x \neq \text{NIL}$

Inorder-Tree-Walk($x.\text{left}$)

Print $x.\text{key}$

Inorder-Tree-Walk($x.\text{right}$)

Complexity: $\Theta(n)$

$$T(n) = T(k) + T(n-k-1) + C$$